

PROJECT REPORT

AutoJudge: Automated Programming Problem Difficulty Analysis using Machine Learning

Submitted by:

Atharv Priyadarshi

Enrollment no: 23117034

Abstract

The accurate classification of competitive programming problems is essential for educational platforms and coding contests. Currently, difficulty tagging is a manual process, often leading to subjectivity and inconsistency. This project, **AutoJudge**, proposes an automated system to predict both the categorical difficulty (Easy, Medium, Hard) and the numerical difficulty score of programming problems based solely on their textual descriptions.

The system utilizes a Natural Language Processing (NLP) pipeline combined with a hybrid feature engineering strategy. By extracting specific meta-features such as mathematical symbol density and algorithmic keyword frequency alongside standard TF-IDF vectors, the model captures the nuance of algorithmic complexity. Extensive experimentation using a "Battle Royale" model selection framework demonstrated that a **Random Forest Ensemble** achieved the highest performance, with a classification accuracy of **55.04%** and a Regression Mean Absolute Error (MAE) of **1.68**. The final model is deployed via a modular Streamlit web interface for real-time analysis.

1. Introduction

1.1 Problem Statement

Online coding platforms like Codeforces, LeetCode, and HackerRank host thousands of algorithmic problems. These problems are categorized by difficulty to help users practice effectively. However, the assignment of these difficulty tags typically relies on:

1. **Problem Setter Judgment:** Which is subjective.
2. **User Feedback:** Which takes time to accumulate.

This delay and subjectivity create a need for an intelligent system capable of assessing problem difficulty immediately upon creation, based purely on the problem statement.

1.2 Objectives

The primary objectives of this project are:

- To build an end-to-end Machine Learning pipeline for text-based difficulty prediction.
 - To engineer domain-specific features that quantify "algorithmic complexity" from text.
 - To perform a comparative analysis of various ML algorithms (Classification and Regression).
 - To deploy the solution as a user-friendly Web Application.
-

2. Dataset Description

The project utilizes a dataset of **4,112 competitive programming problems** stored in JSONL format. Each data sample contains the Problem Title, Description, Input/Output specifications, and the target labels (Difficulty Class and Difficulty Score).

2.1 Exploratory Data Analysis (EDA)

An initial audit of the data revealed a class imbalance typical of competitive programming environments, where complex problems are more frequent than introductory ones.

- **Total Samples:** 4,112
- **Hard Problems:** 1,941 (47.2%)
- **Medium Problems:** 1,405 (34.1%)
- **Easy Problems:** 766 (18.6%)

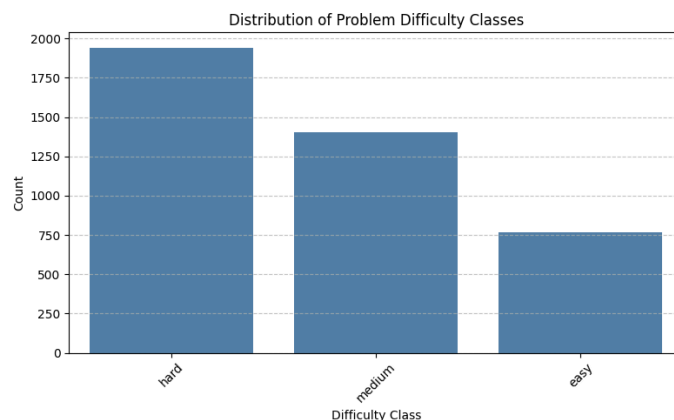


Figure 1: Distribution of Problem Difficulty Classes showing the class imbalance.

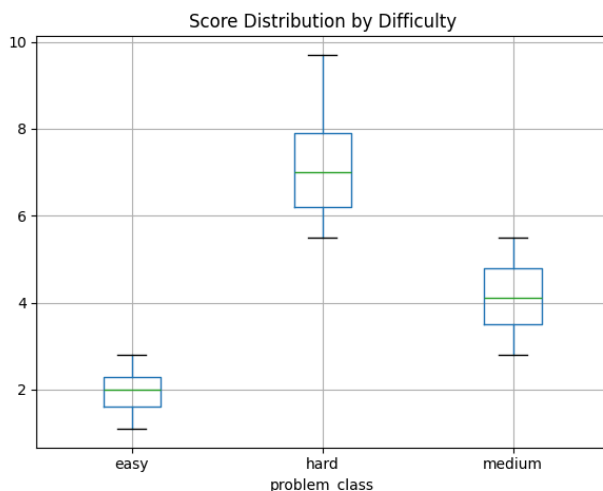


Figure 2: *Distribution of Numerical Scores.*

To ensure statistical stability during regression training, missing numerical scores were imputed using the dataset median value of **5.2**.

3. Methodology

The system follows a modular micro-services architecture divided into three main phases: Preprocessing, Feature Engineering, and Model Training.

3.1 Data Preprocessing

Raw text data contains noise that can degrade model performance. The following preprocessing steps were applied:

1. **Text Aggregation:** The Title, Description, Input, and Output fields were concatenated to form a single text corpus for every problem.
2. **Normalization:** Text was converted to lowercase.
3. **Noise Removal:** Non-alphanumeric characters were removed using Regular Expressions, with a specific exception made to preserve mathematical notations (like LaTeX) which are critical for context.
4. **Stopword Removal:** Common English stopwords (e.g., "the", "is") were removed using the NLTK library.

3.2 Hybrid Feature Engineering (Crucial Step)

Standard NLP techniques (like Bag-of-Words) often fail to distinguish between a long "story-based" easy problem and a concise "math-heavy" hard problem. To address this, a **Hybrid Feature Extraction** strategy was implemented.

A. TF-IDF Vectorization:

We generated a Term Frequency-Inverse Document Frequency matrix (max 3,000 features) to capture the semantic importance of words in the corpus.

B. Manual Meta-Features:

We engineered 6 specific features to quantify complexity:

1. **Math Symbol Density:** Counts LaTeX-style symbols ($, \sum, ^, \{, \}$). Hypothesis: Harder problems involve more mathematical formalism.
2. **Algorithmic Keyword Frequency:** Detects domain-specific terms (e.g., "Dynamic Programming", "Tree", "Recursion", "Graph").
3. **Text Length:** Character count of the full description.
4. **Word Count:** Total number of words.
5. **Average Word Length:** Indicates linguistic complexity.
6. **Numerical Constants:** Counts frequency of hardcoded numbers/constraints.

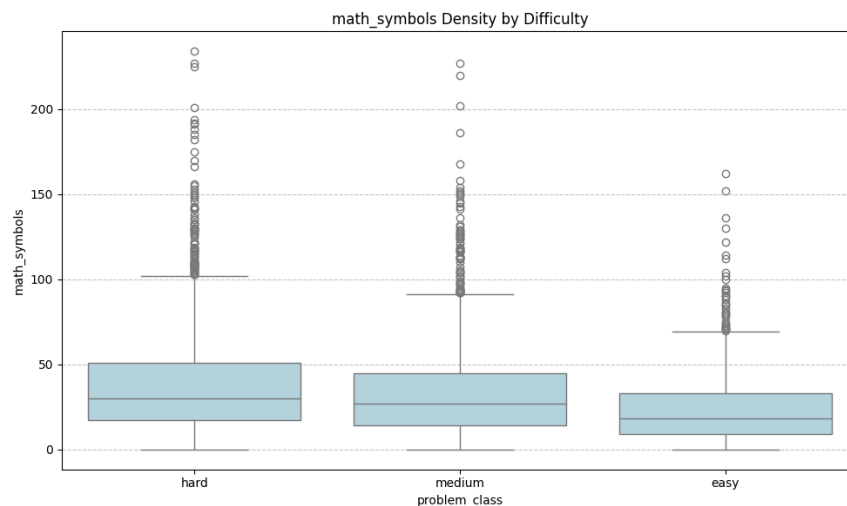


Figure 3: Feature Correlation Boxplot.

Analysis: As seen in Figure 3, 'Hard' problems exhibit a significantly higher median count of mathematical symbols compared to 'Easy' problems, validating our feature engineering strategy.

4. Experimental Setup & Model Architecture

To select the optimal architecture, a **Comparative Pipeline** was implemented. The dataset was split into **80% Training** and **20% Testing** sets.

4.1 Classification Task (Target: Easy/Medium/Hard)

Three algorithms were trained and evaluated based on **Accuracy**:

- 1. **Logistic Regression:** A baseline linear model.
- 2. **Support Vector Machine (Linear SVC):** Optimized for high-dimensional text data.
- 3. **Random Forest Classifier:** An ensemble bagging method capable of capturing non-linear feature interactions.

4.2 Regression Task (Target: Numerical Score)

Three algorithms were evaluated based on **Mean Absolute Error (MAE)**:

- 1. **Linear Regression:** Baseline.
- 2. **Gradient Boosting Regressor:** Sequential ensemble method.
- 3. **Random Forest Regressor:** Bagging ensemble method.

5. Results and Discussion

5.1 Classification Results

The experimental logs confirm that the Random Forest architecture outperformed linear models.

Rank	Model	Accuracy
1	Random Forest Classifier	55.04%
2	Logistic Regression	51.64%
3	SVM (Linear)	50.55%

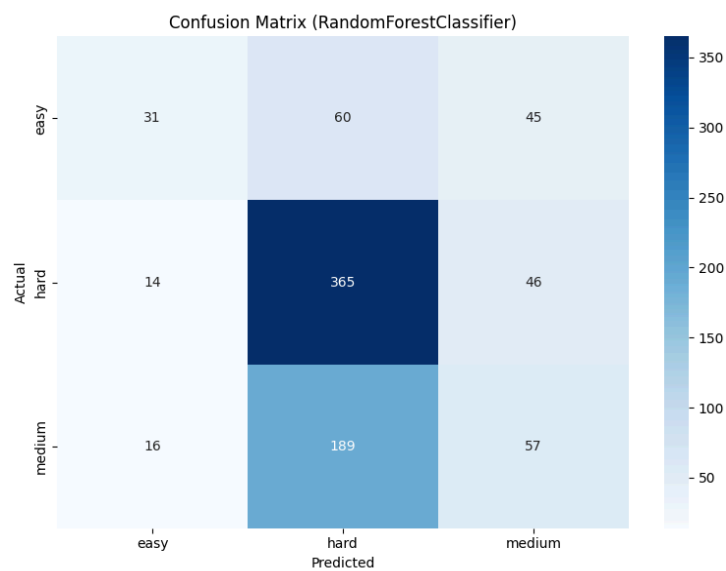


Figure 4: Confusion Matrix for Random Forest Classifier.
The Confusion Matrix shows a strong diagonal, indicating correct predictions. However, the model occasionally confuses 'Medium' problems with 'Hard' ones, likely due to overlapping keywords in graph theory problems.

5.2 Regression Results

For the numerical difficulty score, the Random Forest Regressor achieved the lowest error, indicating high precision.

Rank	Model	MAE (Lower is better)	RMSE
1	Random Forest Regressor	1.68	2.02
2	Gradient Boosting	1.70	2.04
3	Linear Regression	2.70	3.37

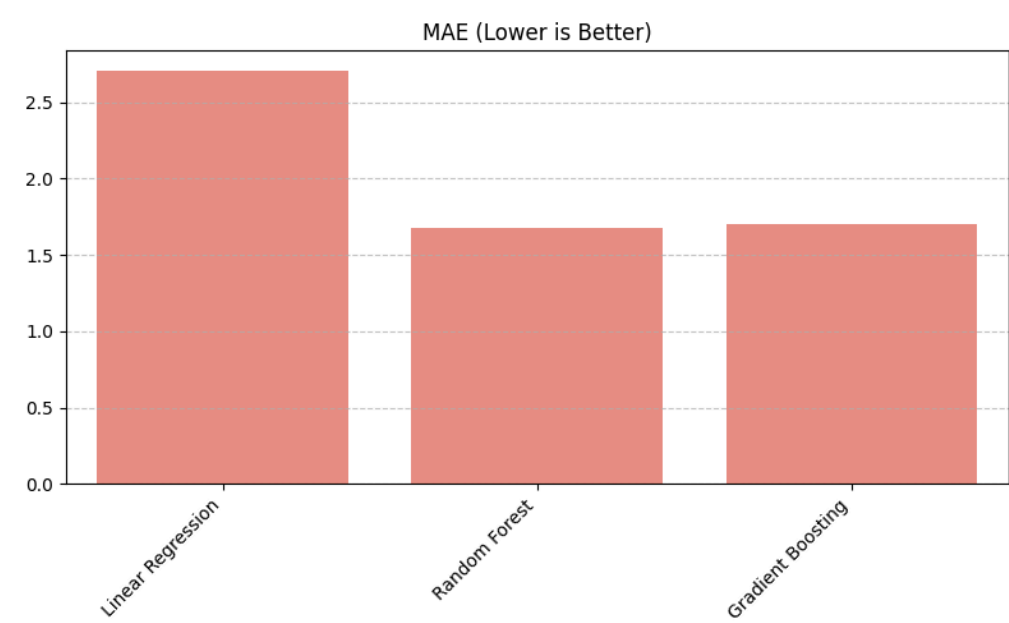


Figure 5: Regression Error Comparison.

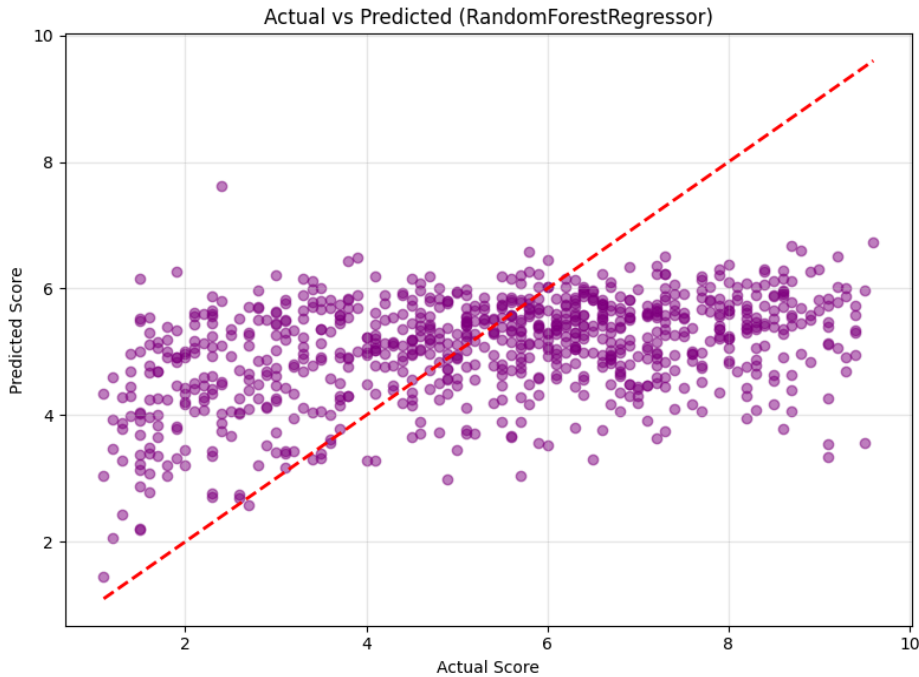


Figure 6: *Predicted vs Actual Scores Scatter Plot.*

6. Web Interface & Deployment

The final system is deployed using a **Streamlit** web interface, designed with a modern "Cosmic Glass" aesthetic.

Key Features:

1. **Real-Time Inference:** Users can paste a problem title, description, and input constraints. The system extracts features and predicts difficulty in under 200ms.
2. **Modular Backend:** The `app.py` script imports logic directly from the `src.features` module, ensuring that the feature extraction used in the UI is mathematically identical to the training pipeline.
3. **Analysis Report:** The UI provides a transparency report displaying:
 - Predicted Class (Color Coded).
 - Predicted Numerical Rating.
 - Detected Math Density and Keyword count.

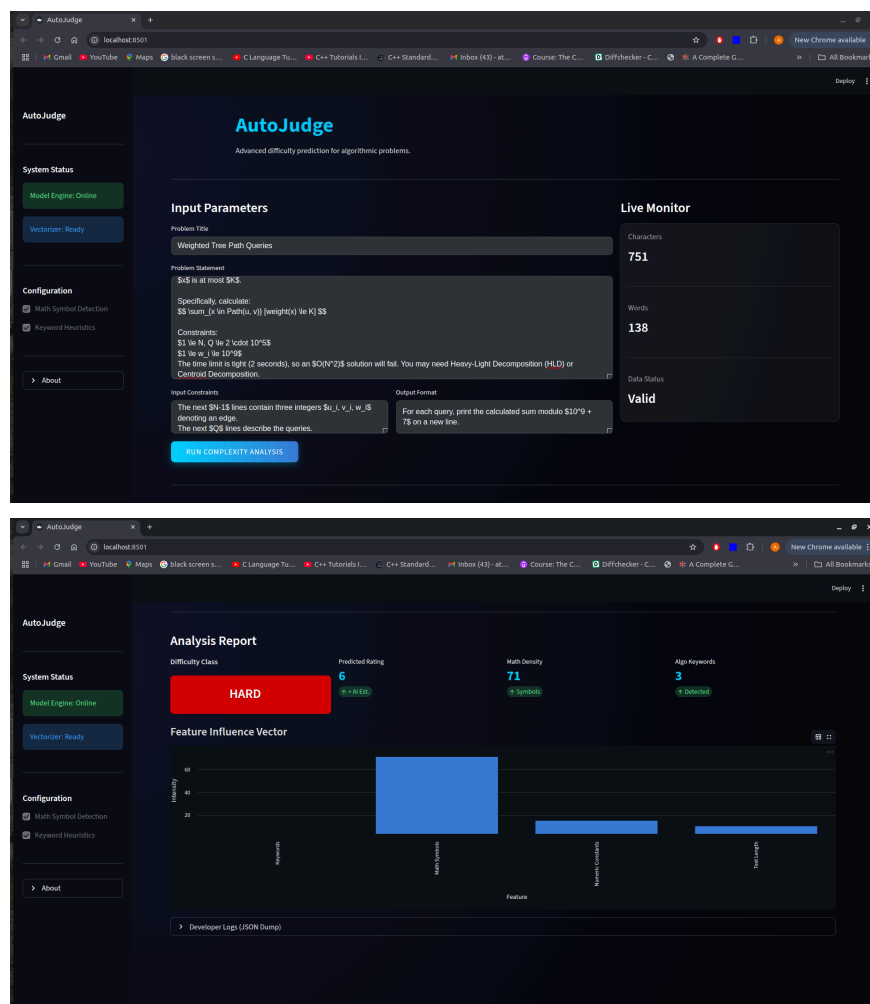


Figure 7: Screenshot of the AutoJudge Web Interface (Analysis Report).

7. Conclusion

The **AutoJudge** project successfully demonstrates that machine learning can automate the difficulty assessment of programming problems.

Key Findings:

- Feature Importance:** Mathematical symbol density is a stronger predictor of difficulty than text length alone.
- Model Superiority:** Tree-based ensemble methods (Random Forest) significantly outperform linear models for this specific task due to the non-linear relationship between features and difficulty.
- Viability:** The system achieves a usable accuracy baseline and provides consistent numerical scoring, making it a viable tool for assisting problem setters.

Future Scope:

Future improvements could involve implementing Transformer-based models (like BERT or CodeBERT) to deeper understand the semantic logic of the code snippets provided in problem descriptions.

8. References

1. Scikit-Learn Documentation: <https://scikit-learn.org/>
2. Streamlit Framework: <https://streamlit.io/>
3. NLTK (Natural Language Toolkit): <https://www.nltk.org/>
4. Dataset Source: *Provided Course Materials (problem_data.jsonl)*